

POLICY FIREWALL

In questo esercizio andiamo a creare una policy firewall che permette alla macchina Kali Linux di effettuare un ping verso la macchina Windows 10 Pro, appartenente alla stessa rete locale precedentemente creata su VirtualBox.

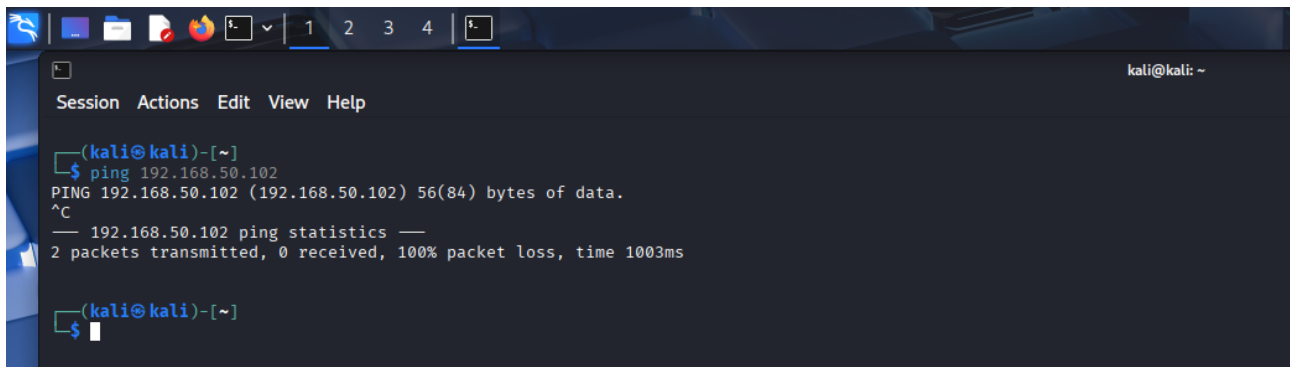
Andiamo nel dettaglio:

dapprima ho verificato che il ping non funzionasse da Kali Linux a Windows (Fig. 1). Una volta accertatomi di ciò, mi sono spostato su Windows ed ho aperto il Firewall di Windows con sicurezza avanzata per impostare una nuova regola personalizzata (Fig. 2).

Successivamente, ho specificato i programmi su cui la regola dovesse avere effetto (Fig. 3), le porte e i protocolli (Fig. 4) e gli indirizzi IP coinvolti (Fig. 5).

Infine, ho impostato l'azione su "Consenti la connessione" (Fig. 6), applicandola a tutti i profili di rete (pubblico, privato e dominio) (Fig. 7).

Terminata la configurazione, mi sono spostato nuovamente su Kali ed ho inviato dei pacchetti ICMP verso la macchina Windows, constatando che li riceveva correttamente: segnale della buona riuscita dell'esercizio. (Fig. 8).



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ ping 192.168.50.102  
PING 192.168.50.102 (192.168.50.102) 56(84) bytes of data.  
^C  
— 192.168.50.102 ping statistics —  
2 packets transmitted, 0 received, 100% packet loss, time 1003ms  
  
(kali@kali)-[~]  
$
```

Figura 1 In questa immagine si nota come Kali non sia capace di inviare pacchetti alla macchina Windows

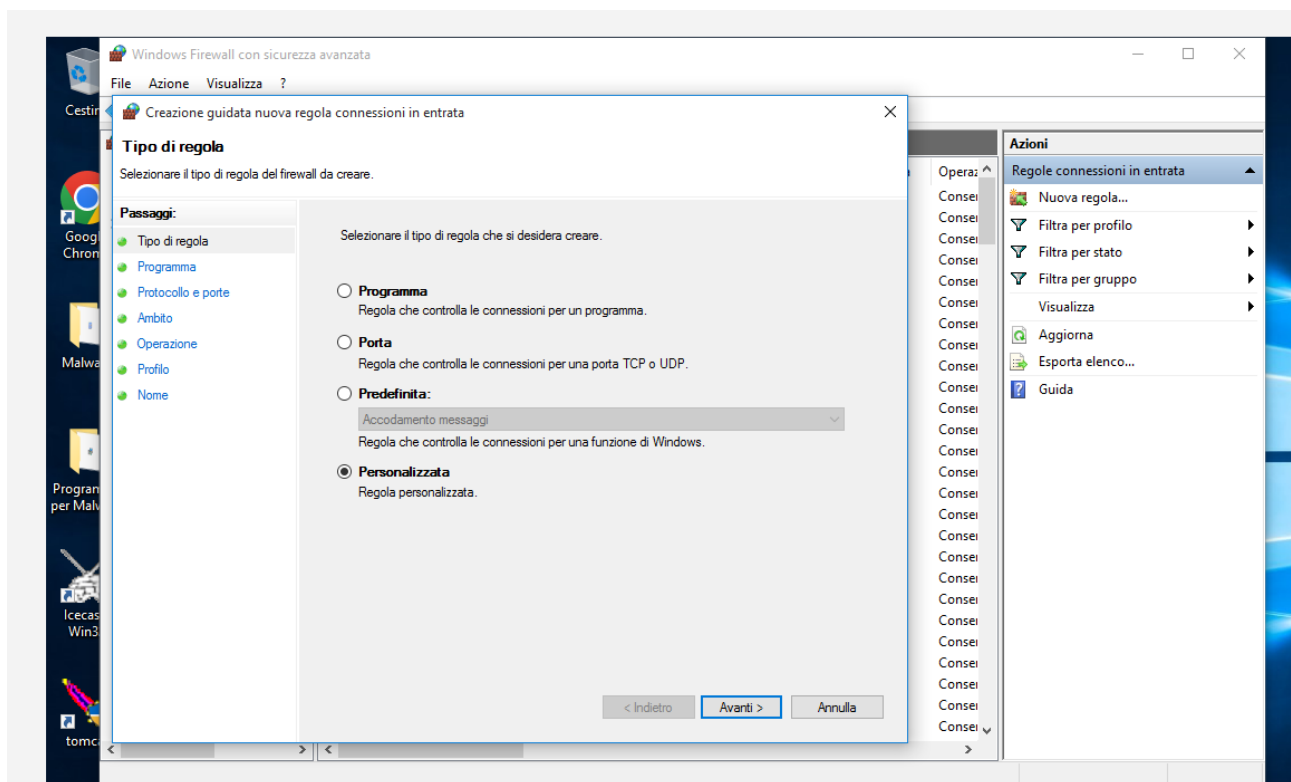


Figura 2 Qui ho iniziato a creare una nuova regola nel firewall di Windows per permettere di ricevere pacchetti da Kali

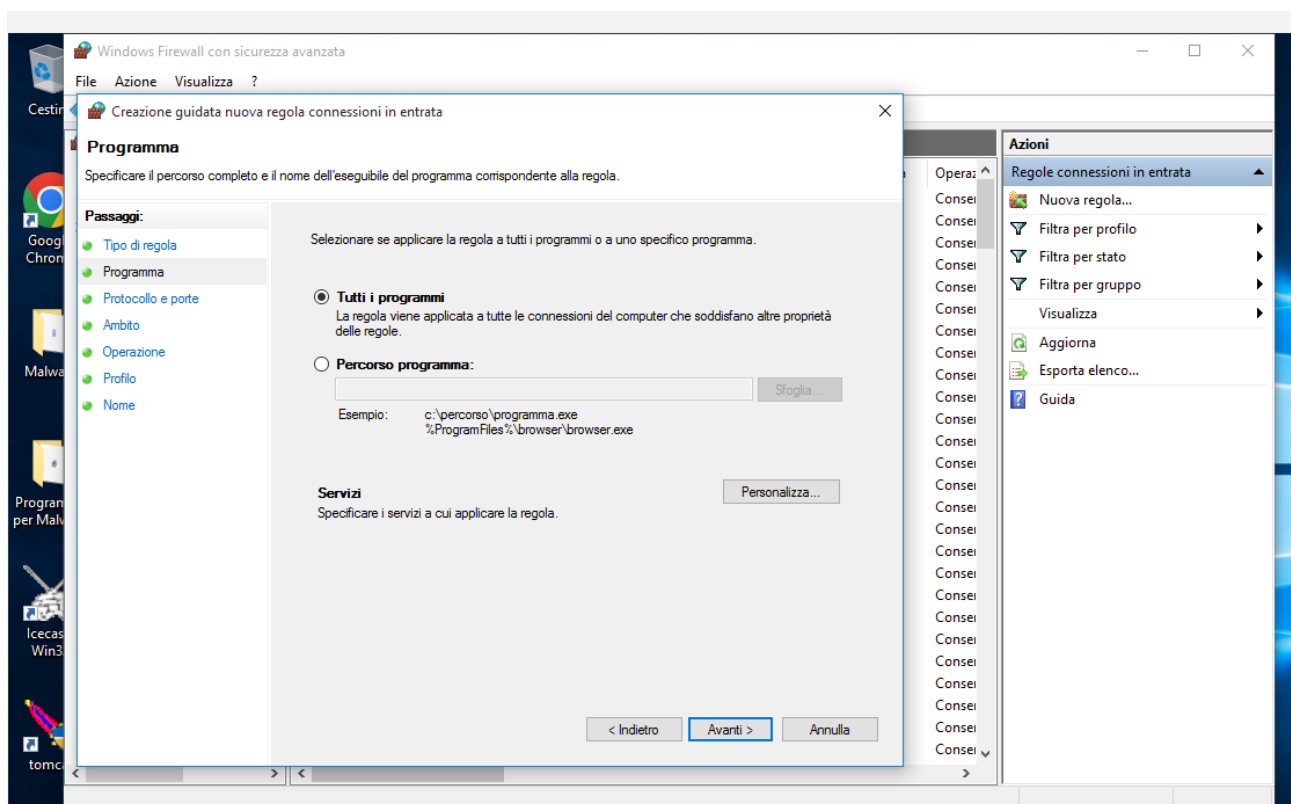


Figura 3 Impostata la regola applicata a tutti i programmi

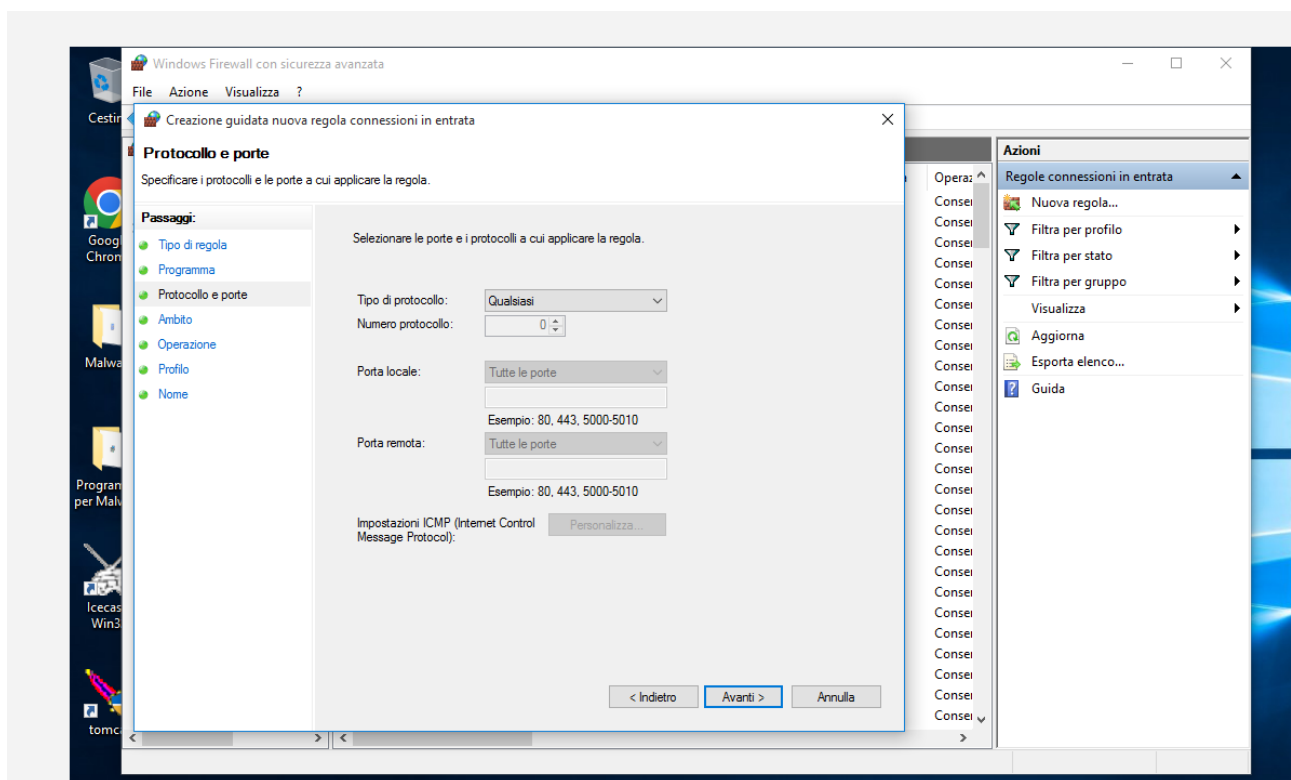


Figura 4 Impostando su qualsiasi tipo di protocollo e su tutte le porte.

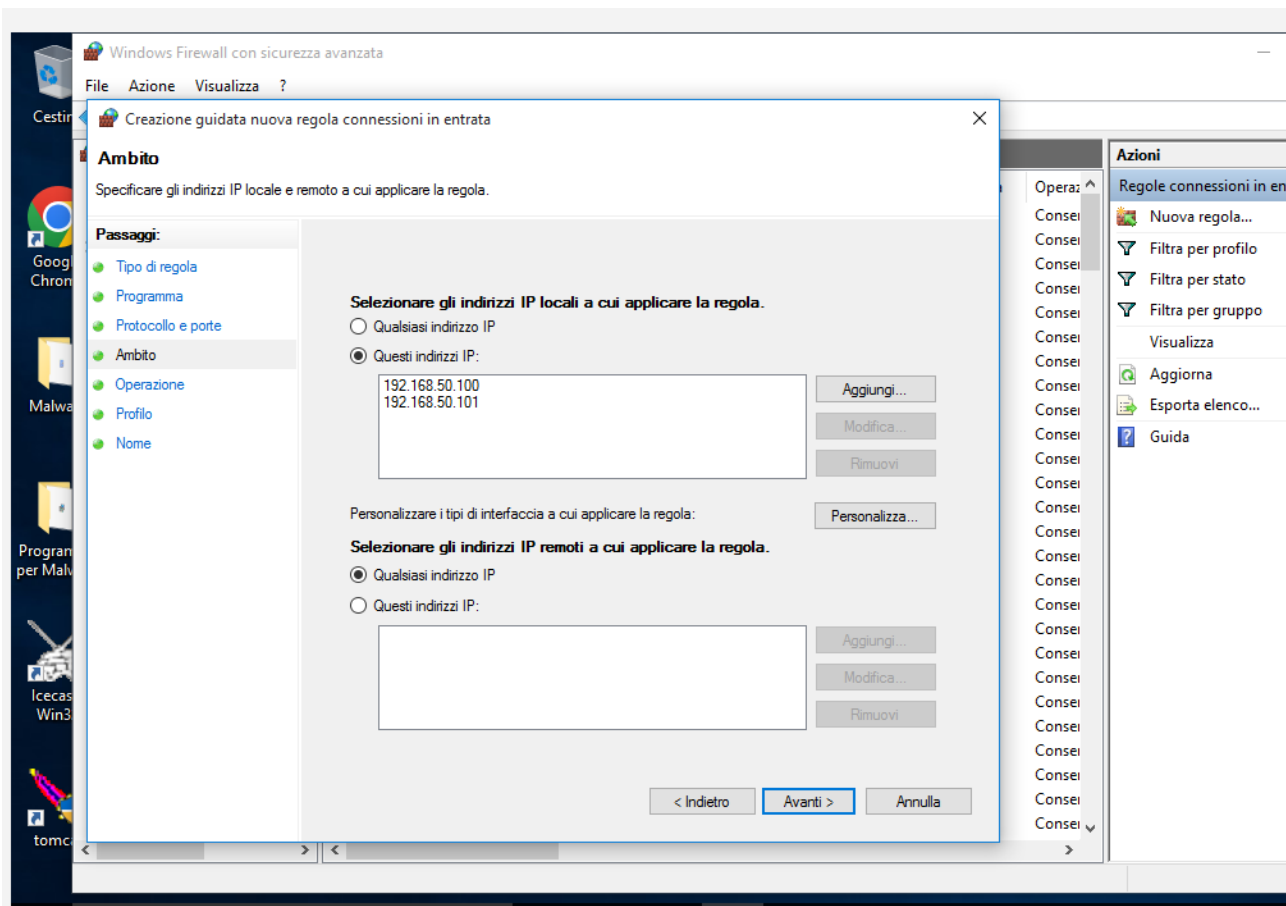


Figura 5 Qui ho inserito gli IP delle macchine virtuali precedentemente create (Kali e Meta2) per permettere di comunicare con Windows.

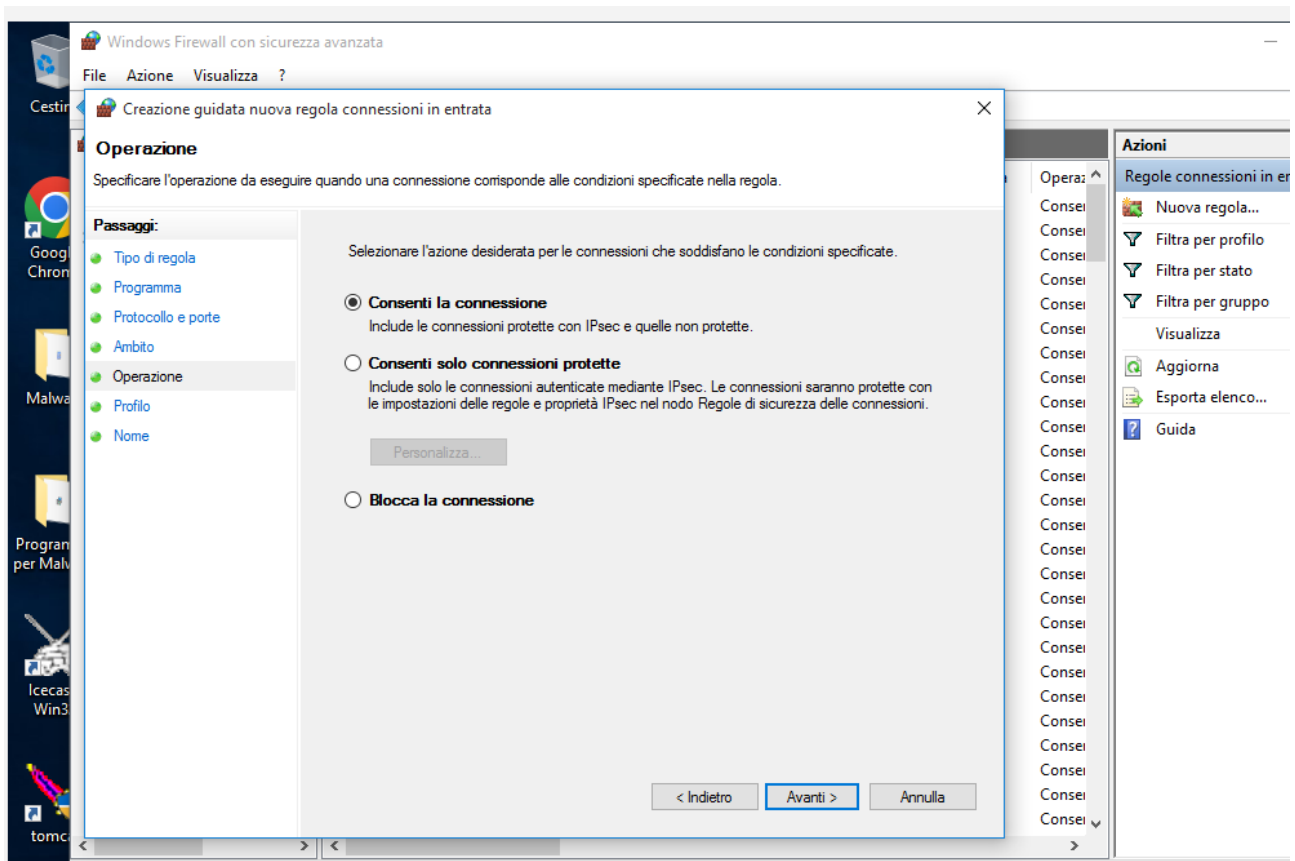


Figura 6 Impostato su *Consenti la connessione* tra le macchine.

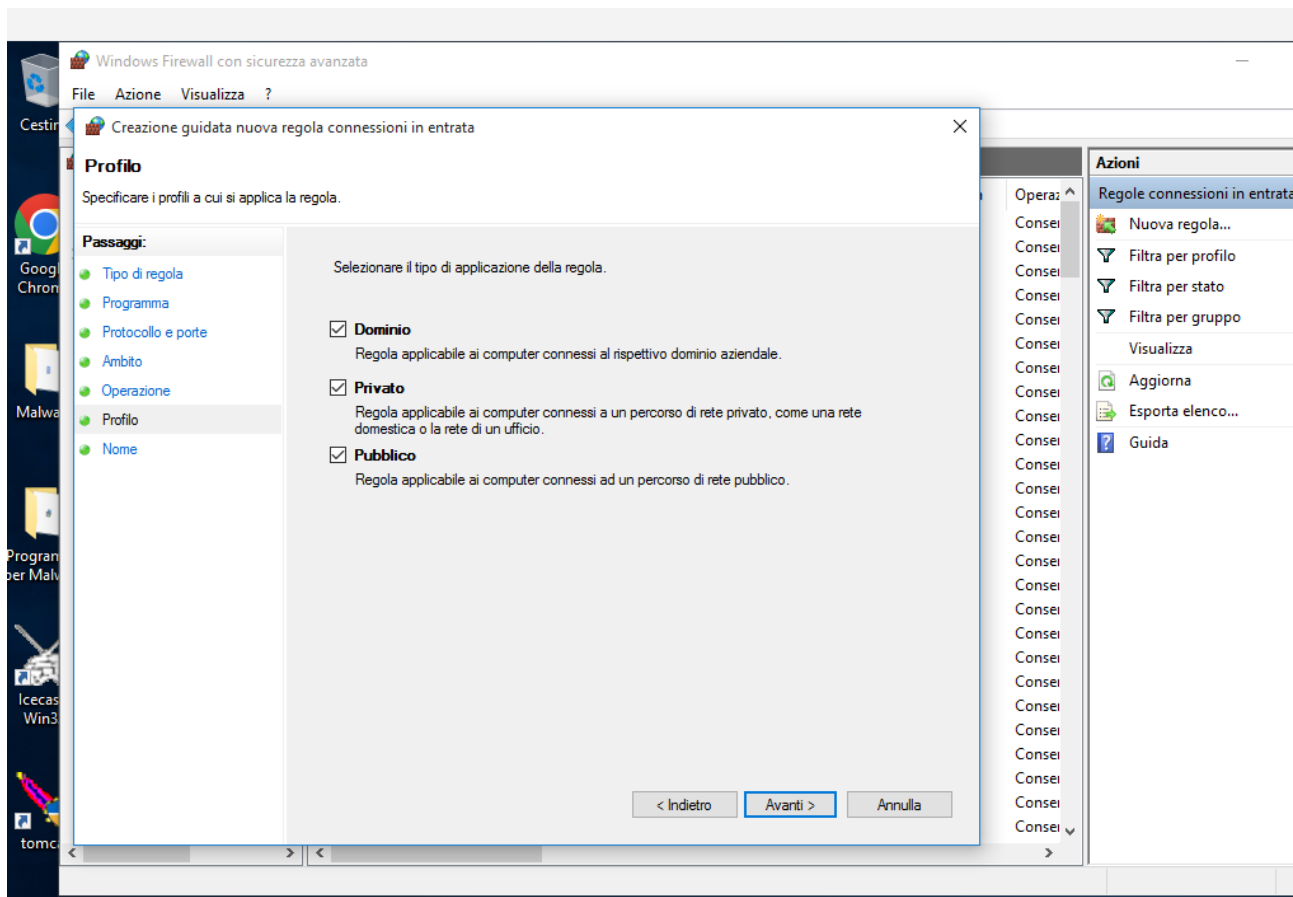


Figura 7 Applicata a tutti i profili.

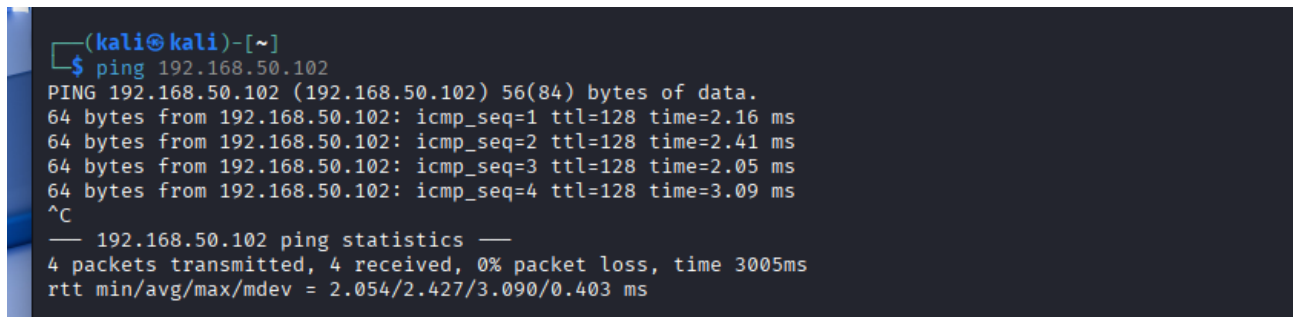


Figura 8 Ora Kali riesce finalmente ad inviare pacchetti a Windows.

Venezia, 09/10/2025

Fabio Renna

INETSIM & WIRESHARK

Questo esercizio utilizza il comando inetsim per emulare il traffico Internet e permette poi di analizzarne i pacchetti con Wireshark.

In primo luogo, su Kali Linux abbiamo configurato i parametri necessari per ridurre il rumore di rete, decidendo di emulare solo il servizio HTTPS (Fig. 1).

Successivamente abbiamo esaminato i “fake files” messi a disposizione da INetSim, utili per simulare risorse web da visitare durante i test (Fig. 2).

Dopo aver selezionato solo il servizio HTTPS, abbiamo avviato INetSim con `sudo inetsim` e messo la macchina in ascolto per avviare la simulazione (Fig. 3).

A questo punto abbiamo aperto Wireshark sull'interfaccia loopback e abbiamo iniziato ad analizzare lo scambio di pacchetti: principalmente traffico TCP, con il relativo handshake (SYN / SYN-ACK / ACK) e i segmenti successivi (ACK, FIN) visibili nelle intestazioni. (Fig. 4)

```
Session Actions Edit View Help
GNU nano 8.6
#####
#
# INetSim configuration file
#
#####

#####
# Main configuration
#####

#####
# start_service
#
# The services to start
#
# Syntax: start_service <service name>
#
# Default: none
#
# Available service names are:
# dns, http, smtp, pop3, tftp, ftp, ntp, time_tcp,
# time_udp, daytime_tcp, daytime_udp, echo_tcp,
# echo_udp, discard_tcp, discard_udp, quotd_tcp,
# quotd_udp, chargen_tcp, chargen_udp, finger,
# ident, syslog, dummy_tcp, dummy_udp, smtps, pop3s,
# ftps, irc, https
#
#start_service dns
#start_service http
start_service https
#start_service smtp
#start_service smtps
#start_service pop3
#start_service pop3s
#start_service ftp
#start_service ftps
#start_service tftp
#start_service irc
#start_service ntp
#start_service finger
#start_service ident
#start_service syslog
#start_service time_tcp
#start_service time_udp
#start_service daytime_tcp
#start_service daytime_udp
```

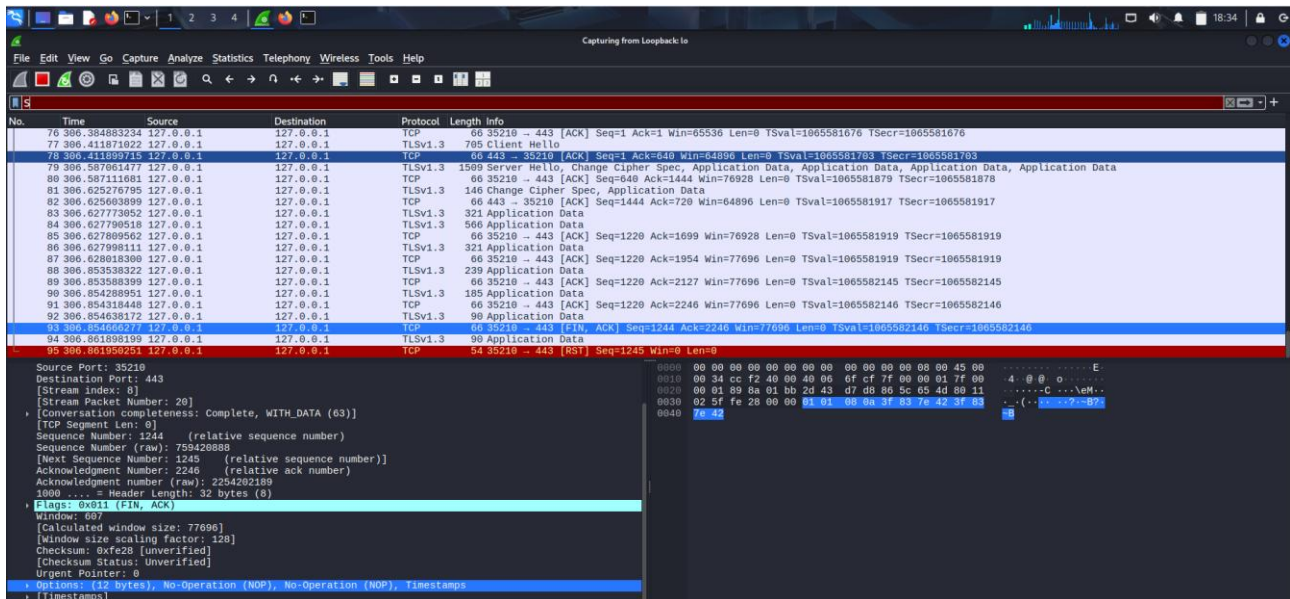
Figura 1 Qui si è tenuto soltanto il servizio https aperto in modo da avere meno traffico simulato.


```
#####
# https_fakefile
#
# Fake files returned in fake mode based on the file extension
# in the HTTPS request.
# The fake files must be placed in <data-dir>/http/fakefiles
#
# Syntax: https_fakefile <extension> <filename> <mime-type>
#
# Default: none
#
https_fakefile txt    sample.txt      text/plain
https_fakefile htm    sample.html      text/html
https_fakefile html   sample.html      text/html
https_fakefile php    sample.html      text/html
https_fakefile gif    sample.gif       image/gif
https_fakefile jpg    sample.jpg       image/jpeg
https_fakefile jpeg   sample.jpg       image/jpeg
https_fakefile png    sample.png       image/png
https_fakefile bmp    sample.bmp       image/x-ms-bmp
https_fakefile ico    favicon.ico     image/x-icon
https_fakefile exe    sample_gui.exe   x-msdos-program
https_fakefile com    sample_gui.exe x-msdos-program
#####
```

Figura 2 in questa figura sono presenti tutti i fake file consultabili.

```
(kali㉿kali)-[~]
└─$ sudo inetsim
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg
Using log directory:      /var/log/inetsim/
Using data directory:     /var/lib/inetsim/
Using report directory:   /var/log/inetsim/report/
Using configuration file: /etc/inetsim/inetsim.conf
Parsing configuration file.
Configuration file parsed successfully.
=== INetSim main process started (PID 20978) ===
Session ID:      20978
Listening on:    127.0.0.1
Real Date/Time:  2025-10-09 18:31:52
Fake Date/Time:  2025-10-09 18:31:52 (Delta: 0 seconds)
Forking services ...
  * https_443_tcp - started (PID 20980)
done.
Simulation running.
```

Figura 3 Qui si è lanciato il comando "sudo inetsim" per avviare la simulazione.



Venezia, 09/10/2025